

Infosys Springboard Virtual Internship 6.0 Completion Report

Batch Number: 13

Start Date: February 02, 2026

Name: Naina Anil Pandey

Internship Duration: 8 Weeks

TABLE OF CONTENTS

Sr No.	Title	Pg No.
1.	Project Title	1
2.	Project Objective	2
3.	Project Description	3-5
4.	Timeline Overview	6
5 a.	Key Milestones	7
5 b.	Project Execution Details	8-9
6.	Screenshots	10-12
7.	Challenges Faced	13
8.	Learning & Skills Acquired	14
9.	Testimonials from Team	15
10.	Conclusion	16
11.	Acknowledgement	17

1. Project Title

Developing an AI System for Personalized Content Creation in Media

This project focuses on building an intelligent AI-based system that can automatically generate customized and high-quality content for different media platforms.

Let's break the title:

➤ **Developing an AI System**

- Refers to designing a software system powered by **Artificial Intelligence**
- Uses **Large Language Models (LLMs)** like Gemini, Groq, etc.
- Capable of understanding user input and generating human-like text

➤ **Personalized Content Creation**

- Content is not generic - it is customized
- Based on user inputs such as:
 - Audience (e.g., Gen Z, CEOs)
 - Tone (e.g., Professional, Casual)
 - Persona (e.g., Marketing Expert)
 - Goal (e.g., Sales, Awareness)
- Example: Same topic → different outputs depending on audience/tone

➤ **In Media:** Refers to real-world platforms where content is used:

- LinkedIn
- Emails
- Advertisements
- Blogs
- Social Media

Why This Title is Relevant?

- Reflects **AI + real-world application**
- Shows personalization (key feature)
- Indicates industry use-case (media/content marketing)
- Aligns with **modern SaaS and AI trends**

2. Project Objective

The primary objective of this project was to design, engineer, and deploy a highly resilient, intelligent content generation platform tailored for media and digital marketing professionals. The goal was to solve the common bottlenecks of manual prompt engineering and AI unreliability by creating a system that abstractly handles complex instructions, brand safety, and multi-constraint generation parameters behind a seamless user interface.

Specifically, the project aimed to:

1. **Automate Contextual Generation:** Enable users to generate hyper-personalized content-ranging from LinkedIn posts and professional emails to ad copy-by simply tuning visual parameters (e.g., Tone, Persona, Target Audience, and Creativity). Reduce manual effort required for writing content. Generate content instantly using AI
2. **Ensure High Availability:** Overcome the rate-limit and downtime issues prevalent in free-tier Large Language Models (LLMs) by engineering a self-healing backend cascade mechanism.
3. **Validate Output Quality:** Implement programmatic Natural Language Processing (NLP) checks to mathematically verify that the AI's output adheres to requested constraints (like keyword inclusion, sentiment, and readability).
4. **Deliver a SaaS-Grade Experience:** Transition from a basic prototype to a fully decoupled, modern web application featuring a highly interactive, responsive, and visually premium interface.
5. **Provide Personalization:** Generate content based on:
 - Target audience
 - Tone of communication
 - Persona (e.g., CEO, Marketer)
 - Goal (e.g., Sales, Awareness)
 - This ensures relevant and engaging output
6. **Enhance User Experience**
 - Build an interactive UI using React
 - Provide: Dashboard, History tracking, Export options (PDF, DOC), Real-time feedback

3. Project Description

The project “**Developing an AI System for Personalized Content Creation in Media**” is a full-stack AI-powered application that generates customized content for different professional and media platforms. The project evolved into a sophisticated, decoupled Software-as-a-Service (SaaS) architecture named "Manifest Core." It combines Frontend (User Interface), Backend (API + Logic) & AI Engine (LLMs + Prompt Engineering). This improves scalability, performance & maintainability

➤ **Approach & Technology:** The backend is built on Python using Flask and LangChain, utilizing Google's Gemini 2.5 Flash as the primary cognitive engine. To ensure zero downtime, a custom "Try-Except" cascade mechanism was engineered to automatically reroute queries to fallback models if API limits are hit. The backend also features rigorous NLP validation algorithms (checking sentiment, readability, and keyword density) and local JSON data persistence. The frontend is a decoupled Single Page Application (SPA) built with React and Vite, styled with Tailwind CSS (v4) and Framer Motion to deliver a premium, glassmorphic User Experience (UX).

➤ **Real-World Impact:** This system significantly accelerates the content lifecycle for digital marketers and content creators. By enforcing brand safety rules and automatically adjusting tone/persona parameters, it reduces the time spent drafting and editing professional communications while ensuring consistent quality.

➤ **Overall Working of the System**

The system follows this workflow:

1. User enters a topic or idea
2. User selects parameters (tone, audience, persona, etc.)
3. Request is sent to backend
4. AI model generates content
5. System validates and processes output
6. Final content is displayed to the user

➤ **Main Components of the System**

1. Frontend (User Interface): Developed using: React.js, Vite, Tailwind CSS, Framer Motion

Features Implemented:

a) Authentication System (Auth.tsx): Ensures secure user access

- Google Login using Firebase
- Email/Password login
- JWT token handling
- Remember Me (localStorage / sessionStorage)
- Forgot password feature

b) Landing Page (Landing.tsx): Improves user engagement

- Modern UI with animations
- Live content preview
- Dark/Light mode
- Marketing-focused design

c) Main Dashboard (App.jsx): Acts as a control panel for AI content generation. This is the core part of the application.

Features:

- Input field for topic
- Parameter selection:
 - Audience
 - Tone
 - Persona
 - Format
 - CTA
- A/B Testing (2 outputs)
- History tracking
- Export options:
 - PDF
 - Word
 - Copy

2. Backend (FastAPI Server): Developed using: FastAPI (Python)

a) Key APIs:

- /api/login → User login
- /api/signup → User registration
- /api/generate → Content generation
- /api/history → Manage history
- /api/prefs → Save preferences

b) Backend Responsibilities:

- Process user input
- Manage authentication
- Handle API requests
- Connect with AI models
- Return structured output

3. AI Engine (Core Intelligence): This is the brain of the system. Ensures reliability, availability & reduced failure

- **Multi-Model Cascade System:**
 - Gemini → Primary
 - Groq → Backup
 - Ollama → Fallback
- **Prompt Engineering System**
 - Uses structured templates for: LinkedIn posts, Emails, Ads
 - Content Customization
 - Based on: Tone, Audience, Persona, Goal, Keywords
- **Output Analysis: Adds intelligence + validation**
System also provides: Word count, Sentiment, Readability & Keywords found

➤ **Key Functionalities**

1. AI Content Generation: Generates high-quality content instantly

2. Personalization Engine: Customizes output based on user inputs

3. A/B Testing: Generates 2 variations for comparison

4. History Management

- Stores previously generated content
- Allows reuse and tracking

5. Export System

- Download content as PDF & Word
- Copy rich text

6. Real-Time Feedback

- Shows: Metrics, Status, Notifications

➤ **Real-World Applications**

- Digital marketing
- Content creation agencies
- Social media management
- Email marketing
- Personal branding

4. Timeline Overview

Weeks	Activities Planned	Activities Completed
Week 1	Project Kickoff, Understand project requirements, Research AI models and tools, API & LLM exploration	Explored 6+ LLM platforms, tested free vs. paid tier limits, and established basic API connections.
Week 2	Backend modular setup, Design prompts for content generation	Developed content-specific prompt templates and finalized backend modular routing structure. Defined parameters like tone, audience, persona, goals, etc.
Week 3	Gemini 2.5 integration, Fallback implementation, Build backend APIs	Built the LangChain pipeline and successfully implemented the multi-model cascade fallback mechanism: Gemini → Groq → Ollama, Implemented request handling.
Week 4	Content validation & Data persistence, Testing same input on different parameters	Created NLP validation algorithms (WPS, sentiment) and integrated JSON databases for user history, Observed outcomes on different parameters.
Week 5	Frontend Development : Architectural redesign to decoupled system, Authentication System	Transitioned from Streamlit to a React/Vite frontend environment; established Flask CORS. Created landing page & basic dashboard, Implemented Google login (Firebase), Email & password login, Added JWT-based authentication, Stored sessions using localStorage.
Week 6	UI/UX Development & Framework setup, State management & API integration, Advanced Features	Implemented Tailwind CSS v4, dark mode, and designed the initial glassmorphic landing page, Replaced mock frontend data with real asynchronous fetch calls to the Flask backend.
Week 7	Final testing, documentation, and handover	Conducted system tests, resolved final UI bugs, and finalized comprehensive milestone reports.
Week 8	Final demonstration and ppt presentation	The project is in the final stage but not completed yet

5a. Key Milestones

Milestone	Description	Date Achieved
Project Kickoff	Internship started: Project topic assigned, Initial project setup, orientation, and requirement gathering.	February 2, 2026
Milestone 1	Model Exploration: Initial working version of the system, API integration tested, limitations documented, and prompting techniques prototyped.	February 15, 2026
Milestone 2	Backend Engineering: LangChain pipeline, Cascade Fallback engine, and programmatic NLP validation finalized, Fully functional backend system.	March 1, 2026
Milestone 3	Frontend Architecture: Complete UI implementation, Decoupled React/Vite UI built, Tailwind styling applied, and client-server API bridge secured.	March 9, 2026
Feature Integration & Advance Feature Implementation	Connecting frontend with backend. Enhancing system functionality, A/B testing (multiple outputs), History tracking, Export options (PDF, DOC, Copy), Metrics display	March 15, 2026
Final Submission	Final improvements before submission, Final end-to-end testing and final report submission.	March 20, 2026
Project Wrap-Up	Final presentation preparation, and final project demonstration.	March 30, 2026

5b. Project Execution Details

The execution of the project was carried out in a systematic and modular manner, ensuring that each component of the system was developed, tested, and integrated efficiently. The development process followed a step-by-step approach, starting from backend development to frontend integration and finally system optimization.

➤ **Phase 1: Research & Feasibility (Weeks 1-2)**

Execution began with a comparative analysis of leading LLMs to identify the most responsive engine for dynamic content creation. It became immediately apparent that relying on a single free-tier API would result in rate-limit errors during production. Therefore, the execution strategy pivoted from building a simple API wrapper to designing a fault-tolerant, multi-model engine. During this phase, rigorous prompt engineering was conducted to establish baseline templates that consistently forced the AI to respect character limits, tone, and formatting constraints.

➤ **Phase 2: Robust Backend Engineering (Weeks 3-4)**

The core logic was executed in Python. Using the *LangChain* framework, Google's Gemini 2.5 Flash was integrated as the primary cognitive engine. To execute the high-availability requirement, a custom Cascade Engine was programmed. This routing logic catches *RateLimitError* or *Timeout* exceptions and silently redirects the user's payload to a secondary model (e.g., Llama or Groq), ensuring uninterrupted service. Furthermore, execution involved building programmatic validation scripts—moving beyond trusting the AI's output blindly by forcing the text through Python functions that mathematically verify keyword inclusion and calculate readability scores (Words-Per-Sentence).

➤ **Phase 3: Decoupling and UI Modernization (Weeks 5-6)**

The final execution phase focused on transforming the robust backend into a user-friendly SaaS product. The original plan of using *Streamlit* was discarded in favor of a decoupled architecture to allow for premium visual design and better performance. A React SPA was bootstrapped using Vite. Execution here involved:

1. **Security/Networking:** Configuring *flask-cors* on the backend to accept cross-origin POST requests from the React development server.
2. **Interface Engineering:** Utilizing Tailwind CSS v4 and Framer Motion to build an immersive, responsive interface featuring interactive *glassmorphism* and real-time simulated terminals.
3. **State Synchronization:** Implementing React hooks (*useState*, *useEffect*) to capture complex user inputs (sliders, dropdowns) and package them into JSON payloads. The UI was executed to actively listen for backend metadata, updating the user dynamically via toast notifications if the Cascade Engine engaged a fallback model.

➤ **Phase 4: Integration & Testing (Week 7)**

1. **API Integration:** Connected frontend with backend APIs & handled requests and responses
2. **Authentication Flow:** Implemented JWT token-based authentication & managed user sessions
3. **Feature Implementation:** Content generation system, History tracking, Export functionality (PDF, DOC, Copy).
4. **Testing & Debugging:** Fixed frontend-backend issues, improved performance & handled edge cases

➤ **Development Methodology**

The project follows an **Agile-like approach**:

- Continuous development
- Regular testing
- Incremental improvements

➤ **Challenges Faced During Execution**

- Integrating multiple AI models
- Handling API errors
- Managing authentication securely
- Synchronizing frontend and backend

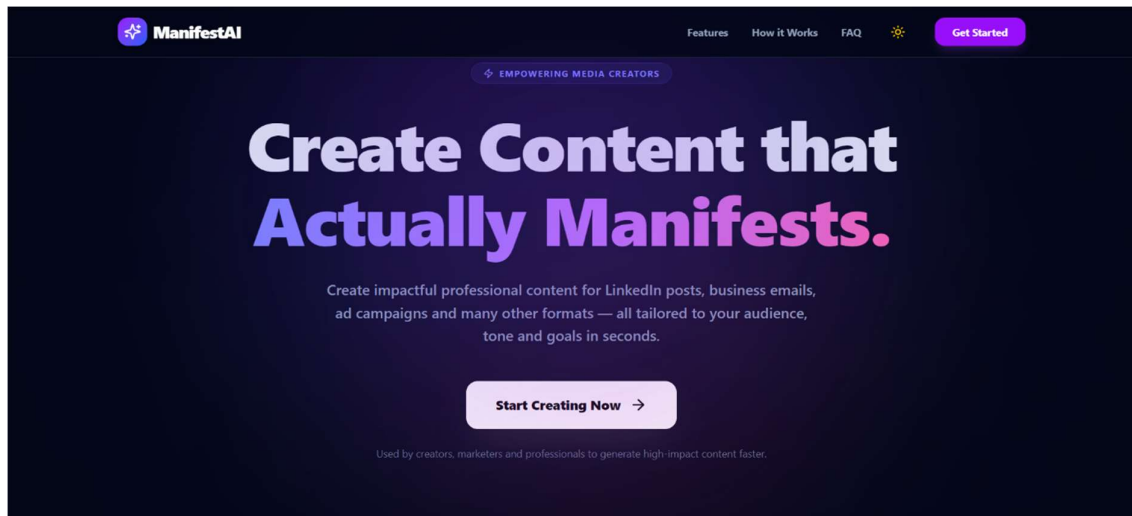
➤ **Solutions Implemented**

- Used cascade system for AI reliability
- Implemented proper error handling
- Used JWT for secure authentication
- Used event-based syncing (auth-change)

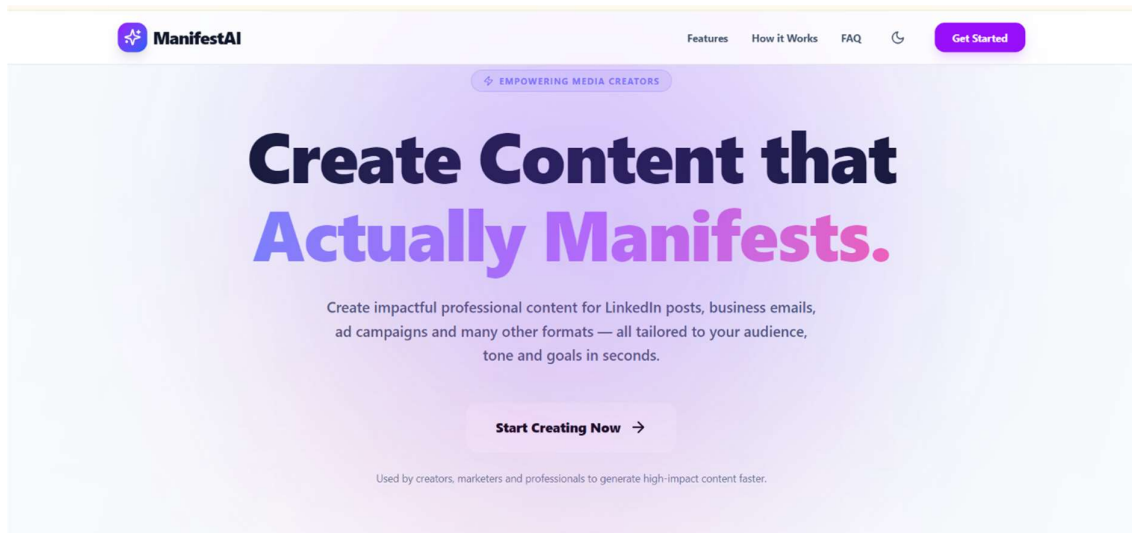
6. Screenshots

Description: The dynamic "Dream. Manifest." Landing Page showcasing the premium glassmorphic UI, animated mesh background, and the interactive Live Terminal Demo.

Dark Mode:

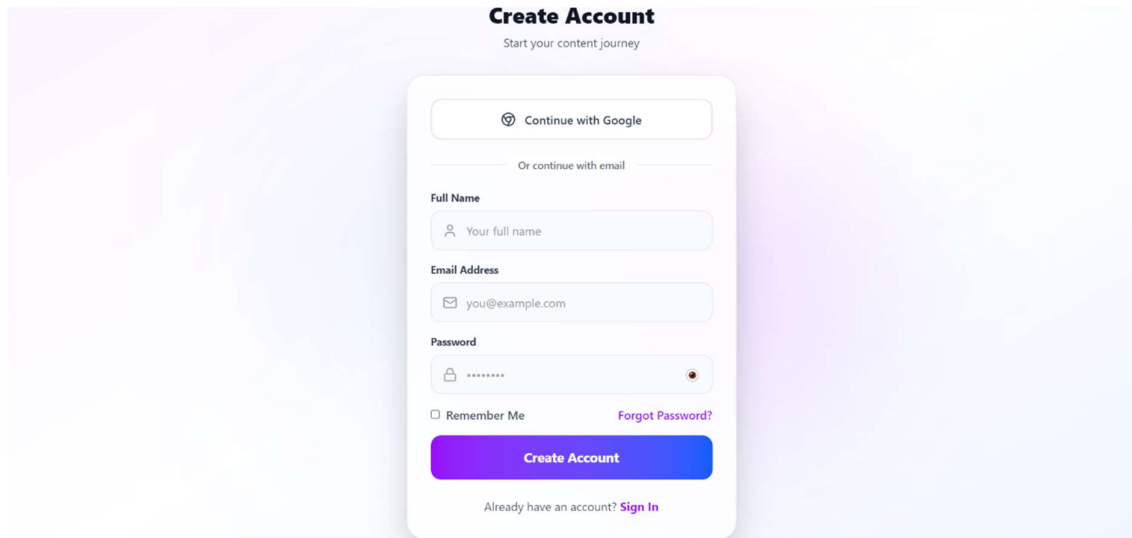


Light Mode:



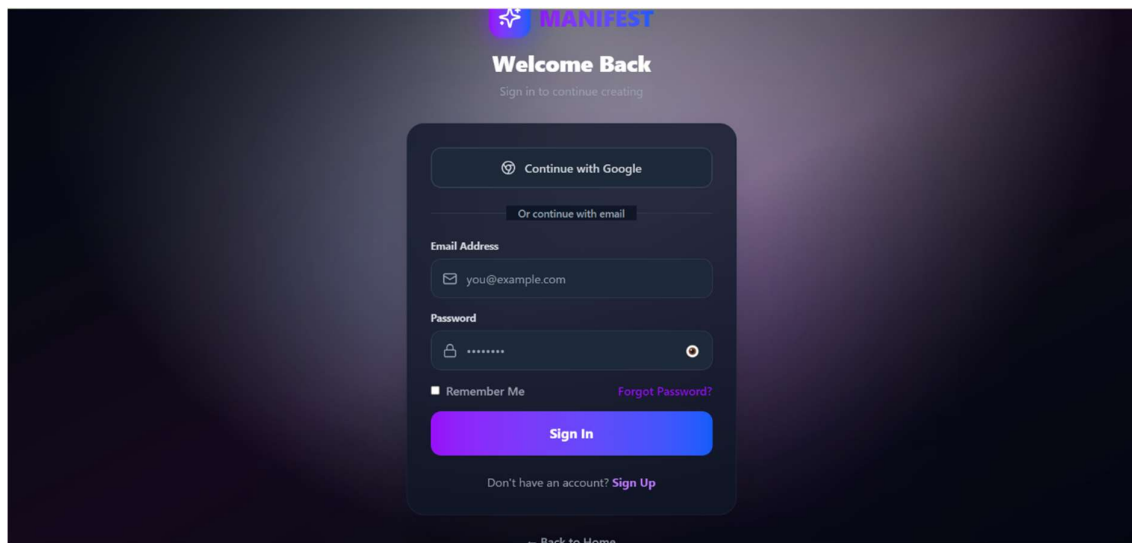
Description: The authentication system asking for User credentials. Signin/ Signup page where user can either create account through their google account or enter name, email id and password. There are also features like remember me and forgot password for user incase of password misplacement.

Light Mode Sign Up Page(Create new users):



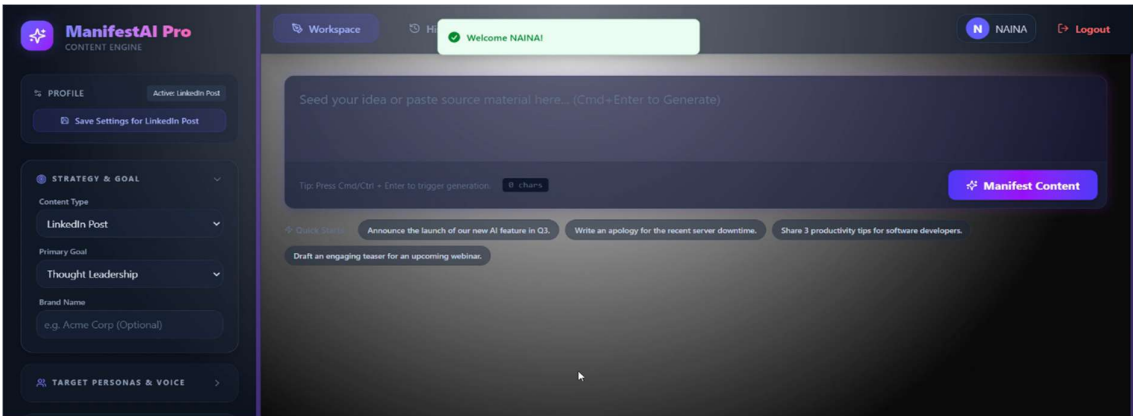
The image shows a 'Create Account' page in light mode. The page has a light purple and blue gradient background. At the top, it says 'Create Account' and 'Start your content journey'. Below this, there is a 'Continue with Google' button. A separator line with the text 'Or continue with email' follows. The form fields are: 'Full Name' (placeholder: 'Your full name'), 'Email Address' (placeholder: 'you@example.com'), and 'Password' (placeholder: '*****'). There is a 'Remember Me' checkbox and a 'Forgot Password?' link. A large blue 'Create Account' button is at the bottom. At the very bottom, it says 'Already have an account? Sign In'.

Dark Mode Sign In Page(For existing users):

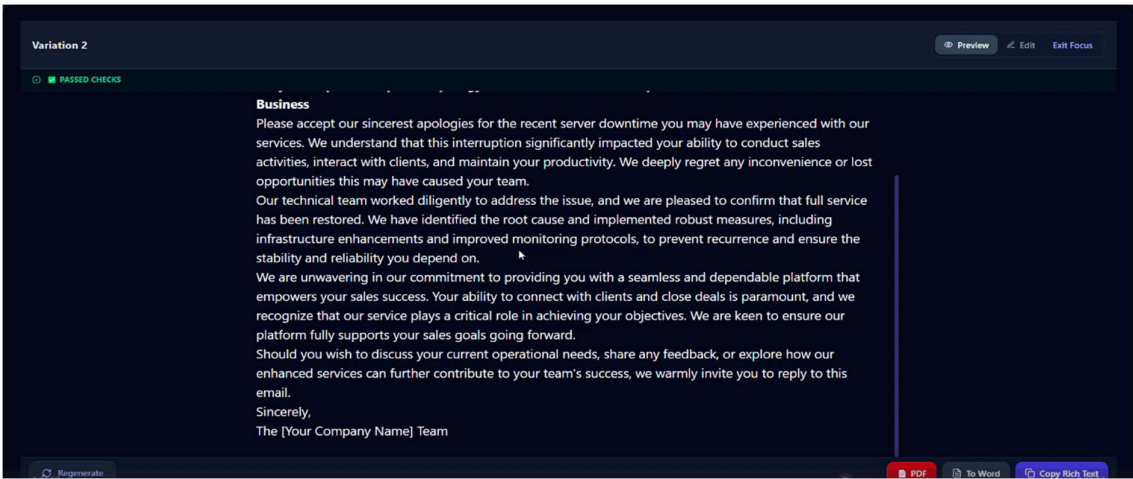


The image shows a 'Welcome Back' sign-in page in dark mode. The background is dark purple and blue. At the top, it says 'Welcome Back' and 'Sign in to continue creating'. Below this, there is a 'Continue with Google' button. A separator line with the text 'Or continue with email' follows. The form fields are: 'Email Address' (placeholder: 'you@example.com'), and 'Password' (placeholder: '*****'). There is a 'Remember Me' checkbox and a 'Forgot Password?' link. A large blue 'Sign In' button is at the bottom. At the very bottom, it says 'Don't have an account? Sign Up'.

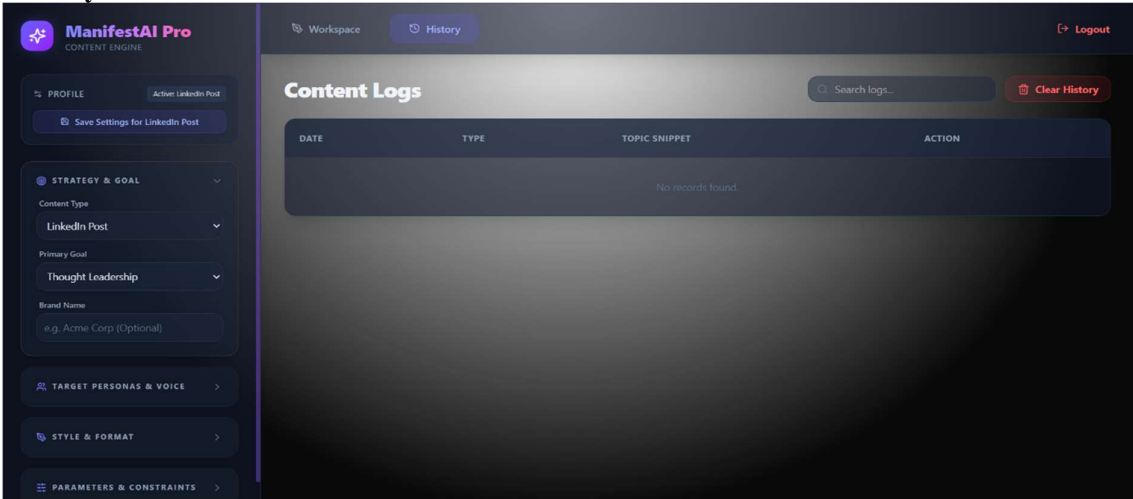
Description: The authenticated user dashboard featuring the Content Archetype Selection grid (LinkedIn, Twitter, Email, Ad Copy). The main generation workspace where users adjust parameters like Tone, Persona, and Creativity (0.0 - 1.0) before dispatching the payload to the Flask backend.



Full Output Mode:



History:



7. Challenges Faced

Developing a decoupled, AI-driven SaaS platform introduced several significant technical and operational challenges:

1. API Rate Limiting & Unreliability:

- *Challenge:* During Phase 1, relying exclusively on free-tier LLM APIs resulted in frequent 429 Too Many Requests and 504 Gateway Timeout errors, completely breaking the user experience.
- *Resolution:* Engineered a robust "Try-Except" cascade mechanism in Python. If the primary model (Gemini 2.5 Flash) fails, the backend seamlessly catches the exception and reroutes the prompt to a secondary model without disrupting the frontend.

2. Integrating Multiple AI Models

- *Challenge:* Different AI models (Gemini, Groq, Ollama) have:
 - Different APIs
 - Different response formats
 - Handling failures was difficult
- *Resolution:* Implemented a cascade (fallback) system:
 - Primary → Gemini
 - Backup → Groq
 - Fallback → Ollama

3. Cross-Origin Resource Sharing (CORS) Blocks:

- *Challenge:* Transitioning from Streamlit (a monolithic framework) to a decoupled React frontend caused strict browser CORS policies to block data transmission between the Vite server (Port 5173) and the Flask API (Port 8000).
- *Resolution:* Integrated and configured the flask-cors middleware within the api.py routing file, explicitly authorizing cross-origin POST requests and enabling secure client-server data flow.

4. Complex State Management:

- *Challenge:* Streamlit handles user state implicitly, but migrating to React required explicitly managing complex, asynchronous user inputs (e.g., waiting for the AI response while simultaneously displaying loading animations).
- *Resolution:* Utilized React hooks (useState, useEffect) to manage component lifecycles, combined with browser localStorage to securely persist user sessions and theme preferences.

5. Authentication & Session Management

- *Challenge:* Managing login securely, keeping users logged in & syncing authentication state across components
- *Resolution:* Used: Firebase Authentication, JWT tokens, localStorage & sessionStorage. Added custom event: auth-change for syncing. Ensures secure and smooth login system

8. Learnings & Skills Acquired

This internship provided profound, hands-on experience bridging artificial intelligence with modern web development. Key takeaways include:

- **Advanced AI Engineering:** Mastered the LangChain framework, advanced prompt engineering, and programmatic LLM control (manipulating temperature, top-p, and system instructions).
- **Full-Stack Architecture:** Gained the ability to design and implement decoupled architectures, effectively connecting a Python/Flask algorithmic backend with a modern JavaScript/React frontend.
- **Modern UI/UX Development:** Acquired deep practical knowledge of utility-first styling with Tailwind CSS v4. Learned to implement complex visual aesthetics, including glassmorphism and Framer Motion component animations, to deliver a commercial-grade user experience.
- **Systems Resilience:** Learned the critical importance of fault-tolerant design by engineering multi-model cascade fallbacks and implementing mathematical NLP validation checks on raw AI outputs.

9. Testimonials from Team

During the course of the internship, the project development involved continuous learning, problem-solving, and implementation of modern technologies. The experience was highly enriching and contributed significantly to both technical and professional growth.

➤ **Experience During Internship**

Working on this project provided hands-on exposure to **real-world software development practices**, including full-stack development, AI integration, and system design.

Some key experiences include:

- Gaining practical knowledge of React and FastAPI integration
- Understanding how to work with AI models and prompt engineering
- Learning to design user-centric interfaces with modern UI/UX principles
- Implementing authentication systems using Firebase and JWT
- Handling real-world challenges such as API failures and debugging

➤ **Team / Mentor Support**

Throughout the internship:

- Guidance was provided in understanding project requirements
- Support was available for resolving technical issues
- Feedback helped improve both code quality and project structure

This support played an important role in successfully completing the project.

➤ **Personal Growth**

This internship helped in:

- Enhancing technical confidence
- Improving logical thinking
- Understanding industry-level project development

➤ **Overall summary:**

Building the Manifest AI engine was a profound journey into full-stack AI development.

Abstracting the complexity of multiple LLMs into a single, intuitive interface taught me the critical balance between powerful backend engineering and accessible user design.

Witnessing the cascade engine successfully fall back during API limits without breaking the UI—and transforming a basic script into a highly animated, React-based SaaS platform—was an incredible technical victory that prepared me for enterprise-level development.

— **Naina Anil Pandey**

10. Conclusion

The "Developing an AI System for Personalized Content Creation in Media" project successfully culminated in a production-ready SaaS prototype. By migrating from a basic Streamlit script to a fully decoupled React and Flask architecture, the application bridges the gap between raw LLM capabilities and practical media generation needs. The system successfully abstracts complex prompt engineering, ensures high availability through fallback routing, and guarantees brand safety via programmatic validation—all delivered through an exceptional, highly responsive user interface. This internship provided invaluable, real-world experience in developing scalable, AI-driven web applications.

➤ **Real-World Impact**

This project can be used in:

- Digital marketing
- Social media management
- Content writing platforms
- Business communication

Helps:

- Save time
- Improve efficiency
- Increase productivity

➤ **Future Scope**

The system can be further improved by:

- Adding more content types (Instagram, YouTube scripts)
- Supporting multiple languages
- Integrating analytics dashboards
- Deploying on cloud platforms
- Adding team collaboration features

➤ **Project Outcome**

The developed system is capable of generating **high-quality content instantly** and providing **personalized outputs** based on:

- Audience
- Tone
- Persona
- Goals

Supporting multiple content formats:

- LinkedIn posts
- Emails
- Ad copy

Delivering additional insights like: Sentiment, readability, word count & total keywords found.

11. Acknowledgements

I would like to express my sincere gratitude to the **Infosys Springboard Virtual Internship 6.0 program** for providing me with an exceptional platform to learn, innovate, and apply my technical knowledge in a real-world context.

I am especially thankful to my mentors and my team for their continuous guidance, valuable insights, and structured approach throughout this 8-week journey. Their support in defining clear development roadmaps and maintaining professional standards was instrumental in shaping this project.

Their encouragement and feedback played a significant role in transforming this project from a basic implementation into a well-structured and comprehensive software solution.

I would also like to acknowledge the learning resources and collaborative environment provided by the program, which greatly contributed to enhancing my technical skills and overall development.